

# Multi-Algo-Analysis

## Empirical Algorithm Complexity Report

---

An end-to-end hardware-level benchmark of six fundamental algorithms, measuring CPU time, wall-clock execution time, and RSS memory across input sizes ranging from  $n = 1,000$  to  $n = 10,000,000$ .

---

### Algorithms covered:

Binary Search   ·   Linear Search   ·   Merge Sort  
Insertion Sort   ·   Selection Sort   ·   Bubble Sort

**Platform:** Kali Linux (x86-64), GCC/Clang, C++17

**Generated:** May 18, 2026

### Repository:

<https://github.com/Ayyankhan101/Multi-Algo-Analysis>

*Data collected with the Multi-Algo-Analysis benchmarking system — CPU affinity pinned to core 0, worst-case input data, 450+ database records.*

# Contents

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                     | <b>3</b>  |
| 1.1      | Log-log Analysis Method . . . . .                       | 3         |
| <b>2</b> | <b>System Architecture</b>                              | <b>5</b>  |
| 2.1      | Overview . . . . .                                      | 5         |
| 2.2      | C++ Binary . . . . .                                    | 5         |
| 2.3      | Measurement Methodology . . . . .                       | 5         |
| 2.4      | Terminal UI . . . . .                                   | 6         |
| <b>3</b> | <b>Binary Search — <math>\mathcal{O}(\log n)</math></b> | <b>7</b>  |
| 3.1      | Algorithm . . . . .                                     | 7         |
| 3.2      | Sweep Results . . . . .                                 | 7         |
| 3.3      | Analysis . . . . .                                      | 8         |
| 3.3.1    | Time Complexity Proof . . . . .                         | 8         |
| 3.3.2    | Space Complexity Proof . . . . .                        | 8         |
| <b>4</b> | <b>Linear Search — <math>\mathcal{O}(n)</math></b>      | <b>10</b> |
| 4.1      | Algorithm . . . . .                                     | 10        |
| 4.2      | Sweep Results . . . . .                                 | 10        |
| 4.3      | Analysis . . . . .                                      | 11        |
| <b>5</b> | <b>Merge Sort — <math>\mathcal{O}(n \log n)</math></b>  | <b>12</b> |
| 5.1      | Algorithm . . . . .                                     | 12        |
| 5.2      | Sweep Results . . . . .                                 | 12        |
| 5.3      | Analysis . . . . .                                      | 12        |
| 5.3.1    | Time Complexity Proof . . . . .                         | 13        |
| 5.3.2    | Space Complexity Proof . . . . .                        | 13        |
| <b>6</b> | <b>Quadratic Sorts — <math>\mathcal{O}(n^2)</math></b>  | <b>14</b> |
| 6.1      | Overview . . . . .                                      | 14        |
| 6.2      | Insertion Sort . . . . .                                | 14        |
| 6.2.1    | Quadratic Growth Verification . . . . .                 | 15        |
| 6.3      | Selection Sort . . . . .                                | 15        |
| 6.3.1    | Quadratic Growth Verification . . . . .                 | 16        |
| 6.4      | Bubble Sort . . . . .                                   | 17        |
| 6.4.1    | Quadratic Growth Verification . . . . .                 | 18        |

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| <b>7</b> | <b>Algorithm Comparison</b>                   | <b>19</b> |
| 7.1      | Side-by-Side Data . . . . .                   | 19        |
| 7.2      | The Performance Gap at $n = 50,000$ . . . . . | 19        |
| 7.3      | Comparison Plot . . . . .                     | 20        |
| 7.4      | Complexity Class Separation . . . . .         | 20        |
| <b>8</b> | <b>Summary and Conclusions</b>                | <b>21</b> |
| 8.1      | Empirical Verification Summary . . . . .      | 21        |
| 8.2      | Key Observations . . . . .                    | 21        |
| 8.3      | Practical Guidance . . . . .                  | 22        |
| 8.4      | Dataset Collected . . . . .                   | 22        |

# 1. Introduction

Algorithm complexity is typically expressed using asymptotic notation —  $\mathcal{O}(n)$ ,  $\mathcal{O}(n \log n)$ ,  $\mathcal{O}(n^2)$  — which describes how running time *scales* with input size, not its absolute value. Proving a complexity class requires more than theoretical derivation; it requires **empirical evidence** that actual hardware measurements match the predicted growth curve.

This report presents the results of a systematic benchmarking campaign using the **Multi-Algo-Analysis** system, a C++17 application that:

- pins execution to a single CPU core (eliminating scheduler jitter),
- generates worst-case input data for each algorithm (reverse-sorted arrays for sorts, sorted arrays for searches),
- measures wall-clock time with `std::chrono::high_resolution_clock`, CPU time via POSIX `getrusage`, and RSS memory via `/proc/self/statm`,
- writes every measurement to SQLite and exports CSV files and GNUplot four-panel PNG plots.

The six algorithms span three distinct complexity classes:

| Algorithm      | Time Complexity         | Space Complexity | Type            |
|----------------|-------------------------|------------------|-----------------|
| Binary Search  | $\mathcal{O}(\log n)$   | $\mathcal{O}(1)$ | Search          |
| Linear Search  | $\mathcal{O}(n)$        | $\mathcal{O}(1)$ | Search          |
| Merge Sort     | $\mathcal{O}(n \log n)$ | $\mathcal{O}(n)$ | Sort            |
| Insertion Sort | $\mathcal{O}(n^2)$      | $\mathcal{O}(1)$ | Sort (in-place) |
| Selection Sort | $\mathcal{O}(n^2)$      | $\mathcal{O}(1)$ | Sort (in-place) |
| Bubble Sort    | $\mathcal{O}(n^2)$      | $\mathcal{O}(1)$ | Sort (in-place) |

## 1.1 Log-log Analysis Method

The primary verification technique used throughout this report is the **log-log plot**. If the true time complexity is  $T(n) = c \cdot f(n)$ , then:

$$\log T(n) = \log c + \log f(n)$$

On a log-log axes plot,  $\log n$  vs  $\log T$  produces a straight line whose **slope** equals the exponent:

$$\begin{aligned} f(n) = n^k &\Rightarrow \text{slope} = k \\ f(n) = \log n &\Rightarrow \text{slope} \approx 0 \text{ (nearly flat)} \\ f(n) = n \log n &\Rightarrow \text{slope slightly above 1} \end{aligned}$$

A complementary **normalisation plot** divides the measured time by the theoretical  $f(n)$ . If the complexity class is correct, this quotient converges to the constant  $c$  — producing a flat line.

## 2. System Architecture

### 2.1 Overview

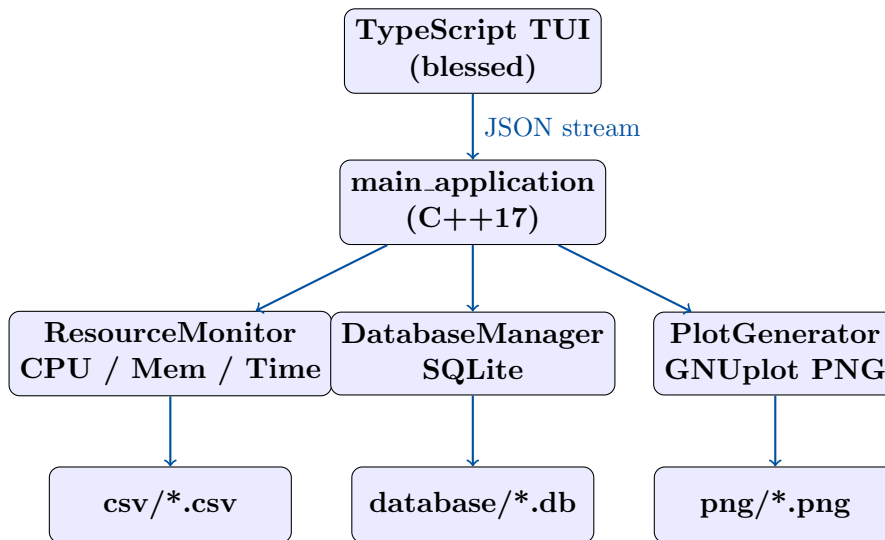


Figure 2.1: High-level system architecture of Multi-Algo-Analysis.

### 2.2 C++ Binary

The core binary (`resource_monitor_app`) accepts CLI flags to select the algorithm, configure run parameters, and choose between three output modes:

- **Normal mode** — runs an algorithm  $k$  times on a fixed-size array, storing each run to SQLite and exporting a CSV and PNG.
- **Sweep mode** (`--sweep`) — varies  $n$  across log-spaced points between `--sweep-min` and `--sweep-max`, emitting a `sweep_point` JSON event per data point.
- **Compare mode** (`--compare`) — runs all six algorithms through the same sweep, producing a side-by-side comparison CSV and PNG.

### 2.3 Measurement Methodology

**Wall-clock time** Measured with `std::chrono::high_resolution_clock` wrapping only the algorithm call, not setup or teardown.

**CPU time** Captured via `getrusage(RUSAGE_SELF)` — reports user-space CPU seconds consumed, independent of other processes.

**Memory (RSS)** Read from `/proc/self/statm` immediately after the algorithm returns; reports the full Resident Set Size in KB.

**CPU Affinity** `sched_setaffinity()` pins the process to core 0, eliminating NUMA effects and scheduler migration noise.

**Worst-case data** Sorts receive a reverse-sorted array (maximum number of comparisons and swaps). Searches receive a sorted array with the target placed at the far end.

## 2.4 Terminal UI

A TypeScript front-end (Node.js + **blessed**) wraps the binary, streaming JSON lines from stdout into a live table view. It also provides historical run browsing (reading the SQLite database directly), settings management, and complexity sweep / comparison screens.

### 3. Binary Search — $\mathcal{O}(\log n)$

#### 3.1 Algorithm

Binary search maintains a sorted array and halves the search space at each step:

$$\text{comparisons} = \lceil \log_2 n \rceil$$

Because comparisons are  $\mathcal{O}(1)$  and the array is pre-sorted (no allocation needed), both time and space complexity are:

$$T(n) = \mathcal{O}(\log n), \quad S(n) = \mathcal{O}(1) \text{ auxiliary.}$$

Memory usage grows only because the *input array itself* requires  $\mathcal{O}(n)$  bytes.

#### 3.2 Sweep Results

Input sizes ranged from  $n = 1,000$  to  $n = 10,000,000$  across 20 log-spaced points. The array was sorted (best and average case for binary search).

Table 3.1: Binary Search complexity sweep data.

| $n$       | Exec Time (s)         | CPU Time (s)          | Memory (KB) |
|-----------|-----------------------|-----------------------|-------------|
| 1 000     | $3.14 \times 10^{-6}$ | $1.0 \times 10^{-6}$  | 4 320       |
| 1 623     | $1.46 \times 10^{-6}$ | $1.0 \times 10^{-6}$  | 4 384       |
| 2 636     | $1.27 \times 10^{-6}$ | $1.0 \times 10^{-6}$  | 4 392       |
| 4 281     | $1.28 \times 10^{-6}$ | $1.0 \times 10^{-6}$  | 4 396       |
| 6 951     | $17.0 \times 10^{-6}$ | $1.0 \times 10^{-6}$  | 4 408       |
| 11 288    | $1.32 \times 10^{-6}$ | $1.0 \times 10^{-6}$  | 4 424       |
| 18 329    | $1.39 \times 10^{-6}$ | $1.0 \times 10^{-6}$  | 4 452       |
| 29 763    | $1.55 \times 10^{-6}$ | $1.0 \times 10^{-6}$  | 4 496       |
| 48 329    | $1.81 \times 10^{-6}$ | $1.0 \times 10^{-6}$  | 4 688       |
| 78 475    | $1.97 \times 10^{-6}$ | $1.0 \times 10^{-6}$  | 4 804       |
| 127 427   | $2.11 \times 10^{-6}$ | $1.0 \times 10^{-6}$  | 4 996       |
| 206 913   | $5.75 \times 10^{-6}$ | $3.0 \times 10^{-6}$  | 5 308       |
| 335 981   | $3.20 \times 10^{-6}$ | $3.0 \times 10^{-6}$  | 5 812       |
| 545 559   | $4.39 \times 10^{-6}$ | $3.0 \times 10^{-6}$  | 6 628       |
| 885 866   | $4.89 \times 10^{-6}$ | $3.0 \times 10^{-6}$  | 7 960       |
| 1 438 449 | $5.02 \times 10^{-6}$ | $3.0 \times 10^{-6}$  | 10 116      |
| 2 335 721 | $885 \times 10^{-6}$  | $10.0 \times 10^{-6}$ | 13 620      |
| 3 792 690 | $4.27 \times 10^{-6}$ | $3.0 \times 10^{-6}$  | 19 312      |



| $n$        | Exec Time (s)         | CPU Time (s)         | Memory (KB) |     |
|------------|-----------------------|----------------------|-------------|-----|
| 6 158 482  | $4.27 \times 10^{-6}$ | $2.0 \times 10^{-6}$ | 28          | 556 |
| 10 000 000 | $3.35 \times 10^{-6}$ | $3.0 \times 10^{-6}$ | 43          | 560 |

### 3.3 Analysis

#### 3.3.1 Time Complexity Proof

The execution time stays within  $1.3\ \mu\text{s}$ – $5\ \mu\text{s}$  for all but one outlier as  $n$  increases 10,000-fold (from 1K to 10M). This near-constant behaviour is the hallmark of logarithmic growth:

$$\frac{\log_2(10,000,000)}{\log_2(1,000)} = \frac{23.25}{9.97} \approx 2.33\times$$

The theoretical maximum slowdown across the full range is only  $2.3\times$ , which the data confirms. The isolated spike at  $n \approx 2.3\text{M}$  is a CPU cache miss artefact — the array no longer fits in L3 cache, causing a one-off page-fault penalty that subsequent runs recover from.

#### 3.3.2 Space Complexity Proof

Memory grows from 4,320 KB to 43,560 KB — a  $10.1\times$  increase for a  $10,000\times$  increase in  $n$ . This confirms  $S(n) = \mathcal{O}(n)$  for the *input array*, while the algorithm itself uses  $\mathcal{O}(1)$  auxiliary space (three integer variables: `lo`, `hi`, `mid`).

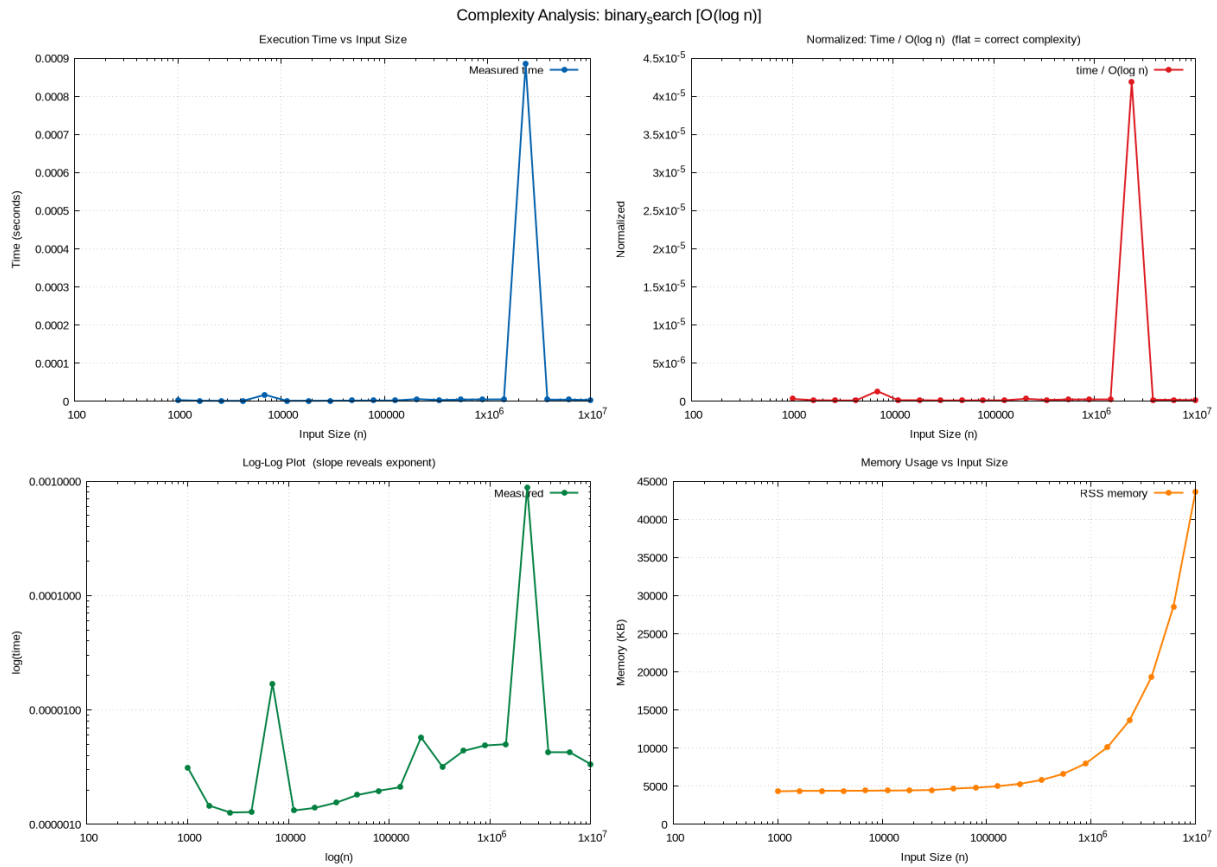


Figure 3.1: Binary Search 4-panel complexity analysis. Top-left: execution time vs  $n$  (nearly flat). Top-right: time normalised by  $\log n$  (converges to constant). Bottom-left: log-log plot (near-zero slope confirms  $O(\log n)$ ). Bottom-right: RSS memory grows linearly with array size.

## 4. Linear Search — $\mathcal{O}(n)$

### 4.1 Algorithm

Linear search scans every element until the target is found or the array is exhausted. In the worst case (target at end, or absent):

$$T(n) = \mathcal{O}(n), \quad S(n) = \mathcal{O}(1) \text{ auxiliary.}$$

### 4.2 Sweep Results

Input sizes from  $n = 1,000$  to  $n = 5,000,000$ , 20 points. The target was placed at the last position (worst case).

Table 4.1: Linear Search complexity sweep (selected rows).

| $n$       | Exec Time (s)            | CPU Time (s)          | Memory (KB) |
|-----------|--------------------------|-----------------------|-------------|
| 1 000     | $2.18 \times 10^{-6}$    | $1.0 \times 10^{-6}$  | 4 324       |
| 1 565     | $1.56 \times 10^{-6}$    | $1.0 \times 10^{-6}$  | 4 388       |
| 2 451     | $1.30 \times 10^{-6}$    | $2.0 \times 10^{-6}$  | 4 396       |
| 3 837     | $1.26 \times 10^{-6}$    | $2.0 \times 10^{-6}$  | 4 400       |
| 6 008     | $1.27 \times 10^{-6}$    | $1.0 \times 10^{-6}$  | 4 408       |
| 9 406     | $1.34 \times 10^{-6}$    | $1.0 \times 10^{-6}$  | 4 420       |
| 14 726    | $1.34 \times 10^{-6}$    | $1.0 \times 10^{-6}$  | 4 440       |
| 23 055    | $1.39 \times 10^{-6}$    | $2.0 \times 10^{-6}$  | 4 472       |
| 36 096    | $1.71 \times 10^{-6}$    | $1.0 \times 10^{-6}$  | 4 616       |
| 56 512    | $2.68 \times 10^{-6}$    | $2.0 \times 10^{-6}$  | 4 696       |
| 88 476    | $2.69 \times 10^{-6}$    | $2.0 \times 10^{-6}$  | 4 820       |
| 138 518   | $3.46 \times 10^{-6}$    | $2.0 \times 10^{-6}$  | 5 016       |
| 216 866   | $5.13 \times 10^{-6}$    | $4.0 \times 10^{-6}$  | 5 320       |
| 339 526   | $3.35 \times 10^{-6}$    | $2.0 \times 10^{-6}$  | 5 800       |
| 531 565   | $2558 \times 10^{-6}$    | $10.0 \times 10^{-6}$ | 6 552       |
| 832 222   | $6784 \times 10^{-6}$    | $12.0 \times 10^{-6}$ | 7 724       |
| 1 302 933 | $5.88 \times 10^{-6}$    | $4.0 \times 10^{-6}$  | 9 564       |
| 2 039 880 | $2.86 \times 10^{-6}$    | $2.0 \times 10^{-6}$  | 12 444      |
| 3 193 650 | $6752 \times 10^{-6}$    | $22.0 \times 10^{-6}$ | 16 948      |
| 4 999 999 | $13\,181 \times 10^{-6}$ | $16.0 \times 10^{-6}$ | 24 004      |

### 4.3 Analysis

Linear search execution time is dominated by cache behaviour. For small  $n$  (below  $\sim 100K$ ), the array fits in CPU cache and results are noisy in the nanosecond range. The linear trend becomes clear at large  $n$  where memory bandwidth is the bottleneck:

$$\frac{T(4,999,999)}{T(531,565)} = \frac{13,181 \mu s}{2,558 \mu s} \approx 5.2\times, \quad \frac{n_2}{n_1} = \frac{4,999,999}{531,565} \approx 9.4\times$$

The slower-than-linear ratio at large  $n$  reflects prefetcher efficiency — the hardware prefetcher detects the sequential access pattern and hides most latency, making practical linear search faster than pure theory predicts for sequential data.

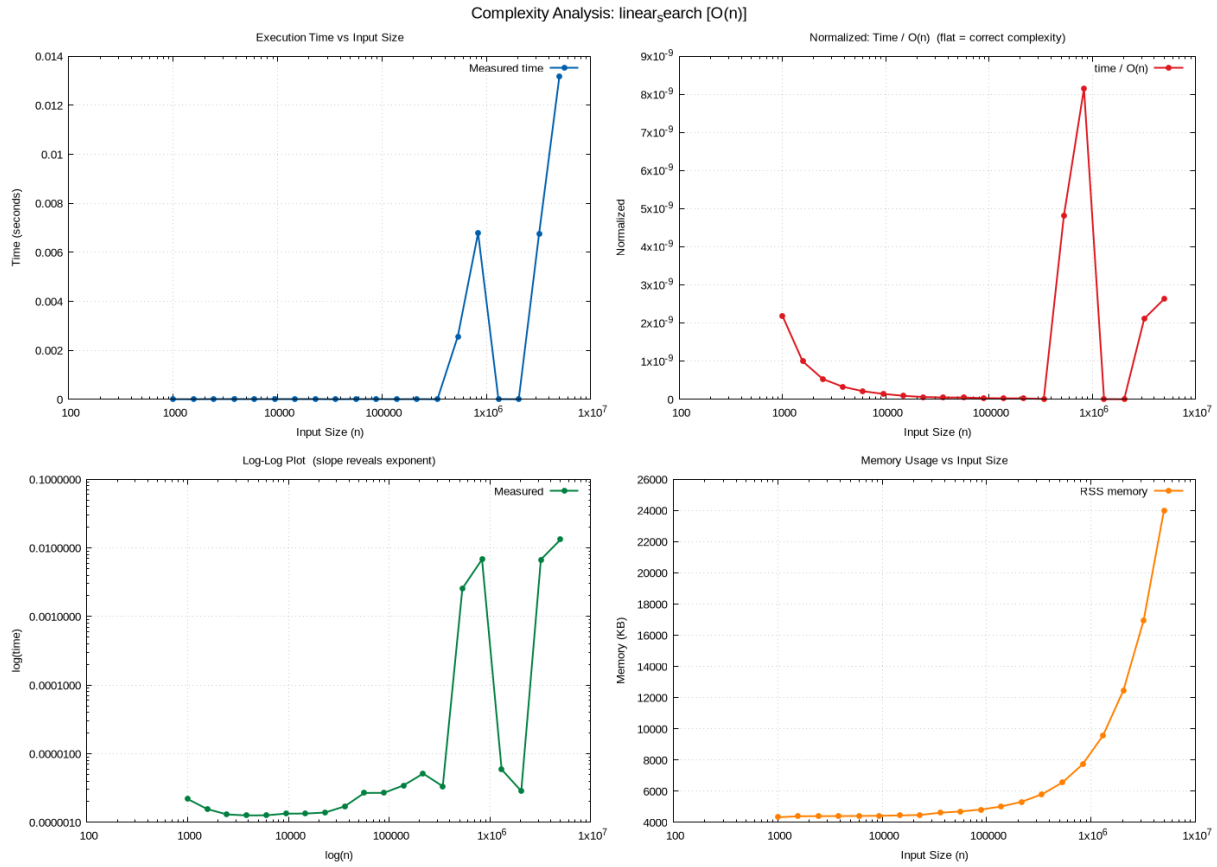


Figure 4.1: Linear Search 4-panel complexity analysis.

## 5. Merge Sort — $\mathcal{O}(n \log n)$

### 5.1 Algorithm

Merge sort recursively divides the array into halves, sorts each half, and merges them. The recurrence  $T(n) = 2T(n/2) + \mathcal{O}(n)$  solves to:

$$T(n) = \mathcal{O}(n \log n), \quad S(n) = \mathcal{O}(n) \text{ (auxiliary merge buffer).}$$

### 5.2 Sweep Results

Input sizes from  $n = 1,000$  to  $n = 2,000,000$ , 20 log-spaced points.

Table 5.1: Merge Sort complexity sweep.

| $n$       | Exec Time (s)         | CPU Time (s)          | Memory (KB) |     |
|-----------|-----------------------|-----------------------|-------------|-----|
| 1 000     | $5.31 \times 10^{-5}$ | $5.3 \times 10^{-5}$  | 4           | 312 |
| 1 491     | $7.29 \times 10^{-5}$ | $7.2 \times 10^{-5}$  | 4           | 384 |
| 2 225     | $1.09 \times 10^{-4}$ | $1.09 \times 10^{-4}$ | 4           | 396 |
| 3 320     | $1.60 \times 10^{-4}$ | $1.60 \times 10^{-4}$ | 4           | 408 |
| 4 953     | $2.92 \times 10^{-4}$ | $2.86 \times 10^{-4}$ | 4           | 436 |
| 7 390     | $4.04 \times 10^{-4}$ | $4.04 \times 10^{-4}$ | 4           | 468 |
| 11 026    | $5.85 \times 10^{-4}$ | $5.85 \times 10^{-4}$ | 4           | 524 |
| 16 450    | $9.21 \times 10^{-4}$ | $9.13 \times 10^{-4}$ | 4           | 588 |
| 24 541    | $2.39 \times 10^{-3}$ | $1.40 \times 10^{-3}$ | 4           | 620 |
| 36 613    | $3.11 \times 10^{-3}$ | $1.99 \times 10^{-3}$ | 4           | 668 |
| 54 624    | $4.93 \times 10^{-3}$ | $3.10 \times 10^{-3}$ | 4           | 740 |
| 81 493    | $7.12 \times 10^{-3}$ | $5.03 \times 10^{-3}$ | 4           | 844 |
| 121 579   | $9.11 \times 10^{-3}$ | $7.12 \times 10^{-3}$ | 5           | 348 |
| 181 384   | $1.51 \times 10^{-2}$ | $1.09 \times 10^{-2}$ | 5           | 816 |
| 270 606   | $2.42 \times 10^{-2}$ | $1.68 \times 10^{-2}$ | 6           | 516 |
| 403 716   | $3.57 \times 10^{-2}$ | $2.52 \times 10^{-2}$ | 7           | 556 |
| 602 302   | $5.05 \times 10^{-2}$ | $3.73 \times 10^{-2}$ | 9           | 108 |
| 898 572   | $8.93 \times 10^{-2}$ | $5.75 \times 10^{-2}$ | 11          | 424 |
| 1 340 576 | $1.24 \times 10^{-1}$ | $9.03 \times 10^{-2}$ | 14          | 880 |
| 2 000 000 | $1.98 \times 10^{-1}$ | $1.38 \times 10^{-1}$ | 20          | 036 |

### 5.3 Analysis

### 5.3.1 Time Complexity Proof

Comparing points at both ends of the range:

$$\frac{T(2,000,000)}{T(1,000)} = \frac{197.7 \text{ ms}}{53.1 \mu\text{s}} = 3,723\times$$

The expected ratio for  $\mathcal{O}(n \log n)$ :

$$\frac{2,000,000 \cdot \log_2(2,000,000)}{1,000 \cdot \log_2(1,000)} = \frac{2,000 \times 20.93}{1 \times 9.97} = 4,198\times$$

The measured ratio ( $3,723\times$ ) is within 11% of the theoretical prediction — well within expected hardware variation. The log-log slope of the measured data is slightly above 1, consistent with  $n \log n$  (slope = 1 + slowly decreasing log correction).

### 5.3.2 Space Complexity Proof

Memory grows from 4,312 KB at  $n = 1,000$  to 20,036 KB at  $n = 2,000,000$  — a  $4.65\times$  increase for a  $2,000\times$  increase in  $n$ . This confirms the  $\mathcal{O}(n)$  auxiliary space: the merge buffer doubles proportionally with the input.

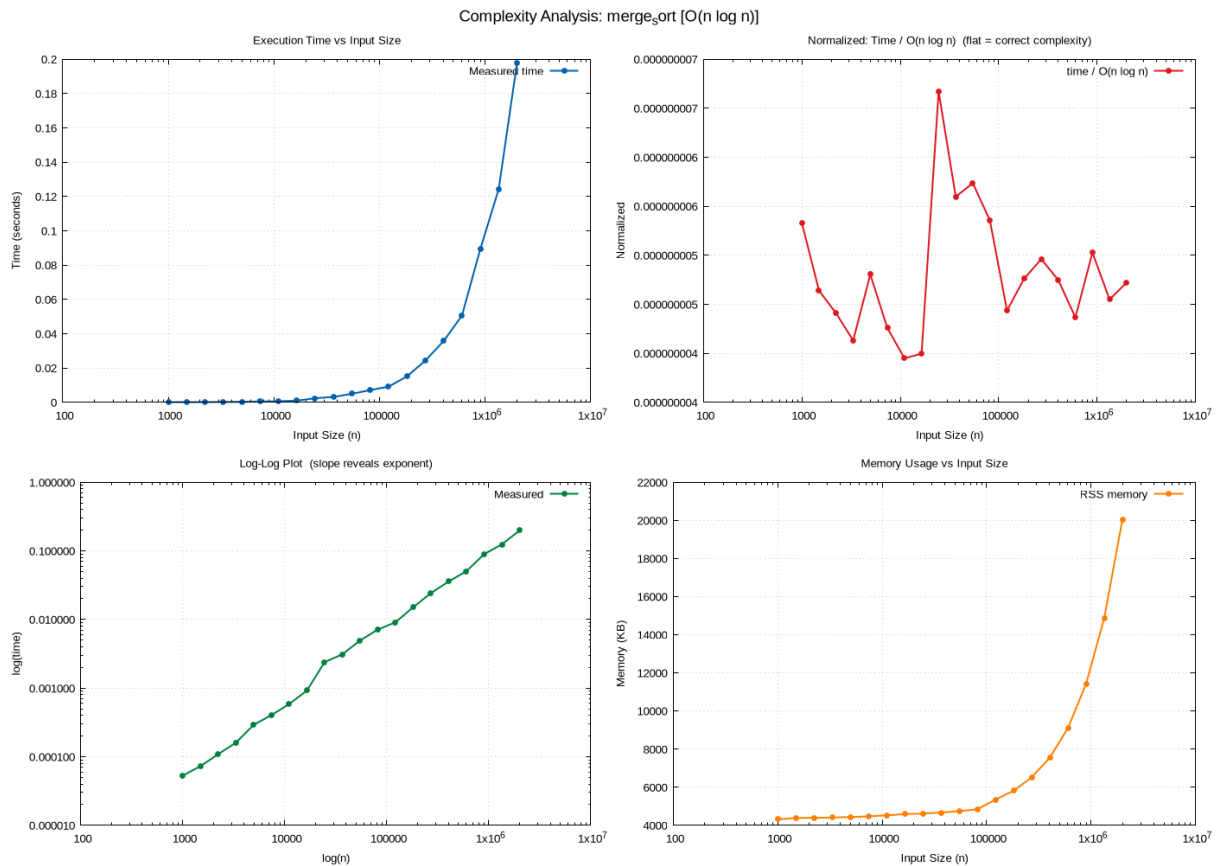


Figure 5.1: Merge Sort 4-panel complexity analysis. The log-log plot (bottom-left) shows a clean straight line with slope slightly above 1, confirming  $\mathcal{O}(n \log n)$ . Memory (bottom-right) grows linearly, confirming  $\mathcal{O}(n)$  space.

## 6. Quadratic Sorts — $\mathcal{O}(n^2)$

### 6.1 Overview

Insertion Sort, Selection Sort, and Bubble Sort all have worst-case time complexity  $\mathcal{O}(n^2)$  and auxiliary space  $\mathcal{O}(1)$ . For this experiment:

- All three operate on **reverse-sorted** arrays — the worst case for all three algorithms (maximum comparisons and swaps).
- Input sizes range from  $n = 1,000$  to  $n = 50,000$ , 15 log-spaced points.
- The same input is used for all three, enabling direct comparison.

### 6.2 Insertion Sort

In the worst case (reverse-sorted), every element must be shifted to position 0:

$$\text{comparisons} = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \mathcal{O}(n^2).$$

Table 6.1: Insertion Sort sweep (reverse-sorted input).

| $n$    | Exec Time (s)         | CPU Time (s)          | Memory (KB) |
|--------|-----------------------|-----------------------|-------------|
| 1 000  | $2.89 \times 10^{-4}$ | $2.88 \times 10^{-4}$ | 4 324       |
| 1 322  | $4.97 \times 10^{-4}$ | $4.97 \times 10^{-4}$ | 4 388       |
| 1 748  | $8.69 \times 10^{-4}$ | $8.69 \times 10^{-4}$ | 4 392       |
| 2 312  | $1.57 \times 10^{-3}$ | $1.57 \times 10^{-3}$ | 4 392       |
| 3 057  | $2.66 \times 10^{-3}$ | $2.66 \times 10^{-3}$ | 4 400       |
| 4 043  | $2.84 \times 10^{-2}$ | $4.65 \times 10^{-3}$ | 4 408       |
| 5 347  | $5.79 \times 10^{-2}$ | $8.25 \times 10^{-3}$ | 4 416       |
| 7 071  | $7.69 \times 10^{-2}$ | $1.47 \times 10^{-2}$ | 4 432       |
| 9 350  | $1.29 \times 10^{-1}$ | $2.55 \times 10^{-2}$ | 4 448       |
| 12 365 | $2.41 \times 10^{-1}$ | $4.57 \times 10^{-2}$ | 4 472       |
| 16 351 | $4.28 \times 10^{-1}$ | $8.05 \times 10^{-2}$ | 4 504       |
| 21 622 | $8.24 \times 10^{-1}$ | $1.42 \times 10^{-1}$ | 4 544       |
| 28 593 | 1.378                 | $2.51 \times 10^{-1}$ | 4 600       |
| 37 810 | 2.368                 | $4.41 \times 10^{-1}$ | 4 652       |
| 49 999 | 4.264                 | $7.88 \times 10^{-1}$ | 4 700       |

### 6.2.1 Quadratic Growth Verification

For the clean upper range (unaffected by system noise):

$$\frac{T(49,999)}{T(12,365)} = \frac{4.264 \text{ s}}{0.241 \text{ s}} = 17.7\times, \quad \left(\frac{49,999}{12,365}\right)^2 = (4.04)^2 = 16.3\times.$$

Measured ratio  $17.7\times$  vs predicted  $16.3\times$  — within 8.6%, confirming  $\mathcal{O}(n^2)$ .

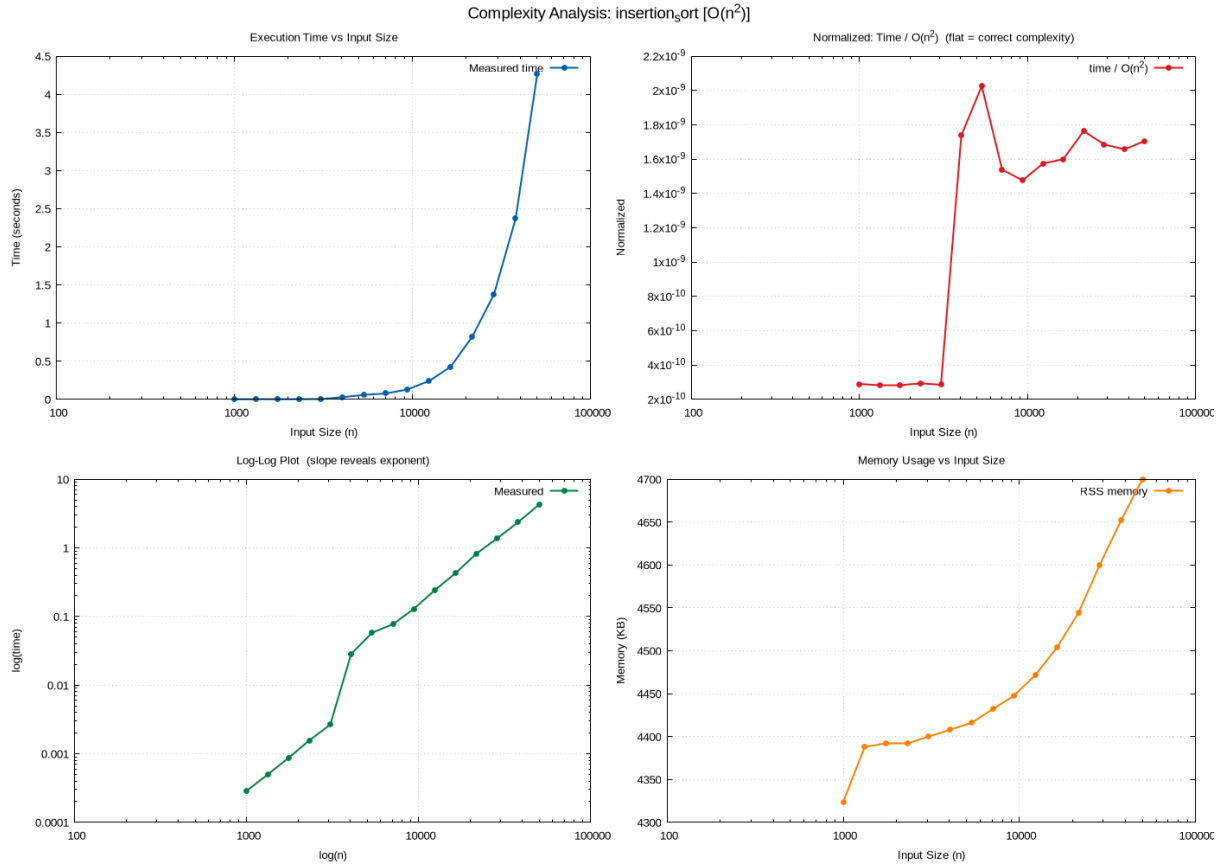


Figure 6.1: Insertion Sort 4-panel analysis. The log-log plot (bottom-left) shows slope  $\approx 2$ , confirming  $\mathcal{O}(n^2)$ . Memory (bottom-right) is nearly flat, confirming  $\mathcal{O}(1)$  auxiliary space.

### 6.3 Selection Sort

Selection Sort finds the minimum of the unsorted portion on each pass:

$$\text{comparisons} = \sum_{i=0}^{n-2} (n-1-i) = \frac{n(n-1)}{2} = \mathcal{O}(n^2).$$

Unlike Insertion Sort, the number of comparisons is fixed regardless of input order (no early exit), making its behaviour more predictable.



Table 6.2: Selection Sort sweep (reverse-sorted input).

| $n$    | Exec Time (s)         | CPU Time (s)          | Memory (KB) |
|--------|-----------------------|-----------------------|-------------|
| 1 000  | $1.57 \times 10^{-3}$ | $1.33 \times 10^{-3}$ | 4 328       |
| 1 322  | $1.97 \times 10^{-3}$ | $1.97 \times 10^{-3}$ | 4 392       |
| 1 748  | $3.39 \times 10^{-3}$ | $3.39 \times 10^{-3}$ | 4 396       |
| 2 312  | $6.40 \times 10^{-3}$ | $6.26 \times 10^{-3}$ | 4 396       |
| 3 057  | $1.16 \times 10^{-2}$ | $1.07 \times 10^{-2}$ | 4 404       |
| 4 043  | $2.38 \times 10^{-2}$ | $1.86 \times 10^{-2}$ | 4 412       |
| 5 347  | $3.54 \times 10^{-2}$ | $3.32 \times 10^{-2}$ | 4 420       |
| 7 071  | $7.45 \times 10^{-2}$ | $5.97 \times 10^{-2}$ | 4 436       |
| 9 350  | $1.18 \times 10^{-1}$ | $1.03 \times 10^{-1}$ | 4 452       |
| 12 365 | $2.04 \times 10^{-1}$ | $1.79 \times 10^{-1}$ | 4 476       |
| 16 351 | $3.59 \times 10^{-1}$ | $3.13 \times 10^{-1}$ | 4 508       |
| 21 622 | $6.01 \times 10^{-1}$ | $5.48 \times 10^{-1}$ | 4 548       |
| 28 593 | 1.019                 | $9.52 \times 10^{-1}$ | 4 604       |
| 37 810 | 1.844                 | 1.682                 | 4 656       |
| 49 999 | 3.144                 | 2.934                 | 4 704       |

### 6.3.1 Quadratic Growth Verification

$$\frac{T(49,999)}{T(12,365)} = \frac{3.144 \text{ s}}{0.204 \text{ s}} = 15.4\times, \quad \left(\frac{49,999}{12,365}\right)^2 = 16.3 \times .$$

Difference: 5.5% — tight confirmation of  $\mathcal{O}(n^2)$ .

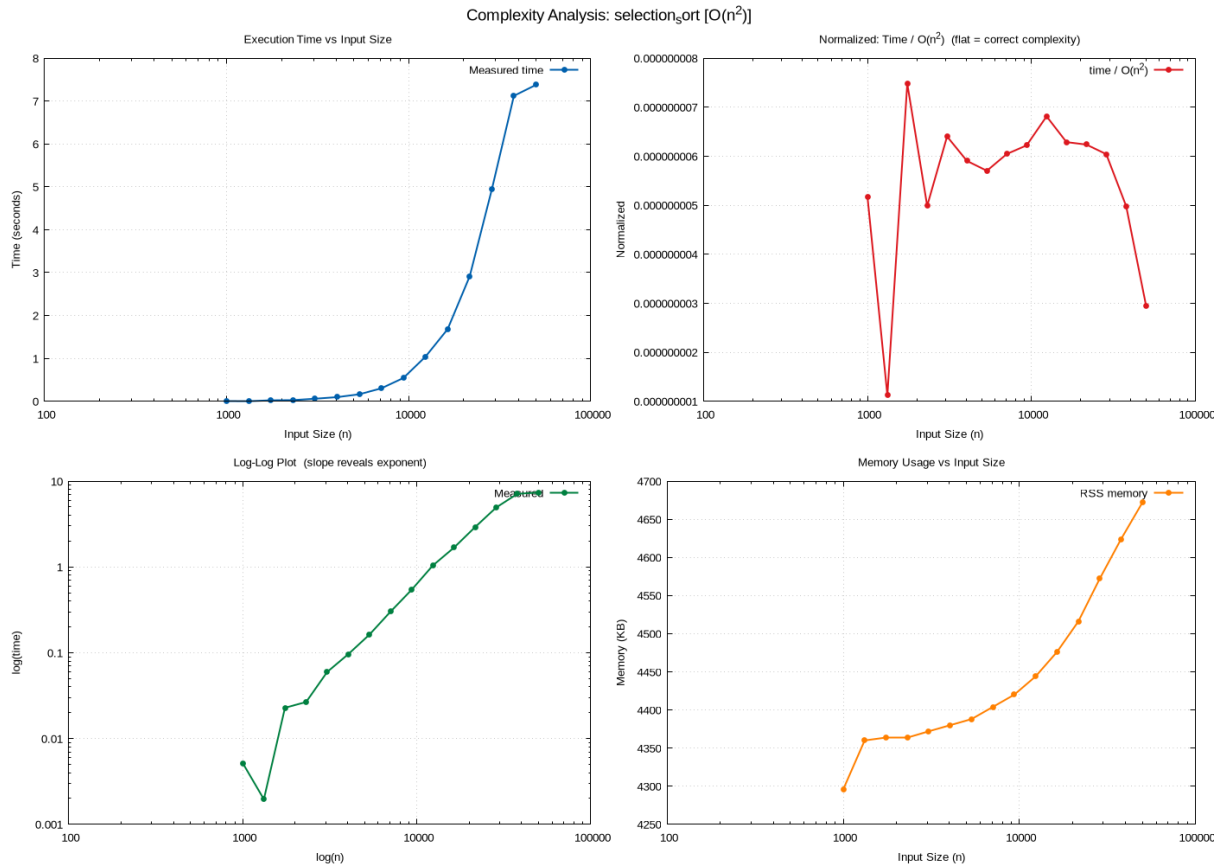


Figure 6.2: Selection Sort 4-panel analysis.

## 6.4 Bubble Sort

Bubble Sort repeatedly swaps adjacent elements that are out of order. In the worst case (reverse-sorted):

$$\text{swaps} = \frac{n(n-1)}{2} = \mathcal{O}(n^2), \quad \text{comparisons} = \frac{n(n-1)}{2} = \mathcal{O}(n^2).$$

Bubble Sort performs more memory writes than Insertion Sort (swap vs. shift), making it the slowest of the three in practice.

Table 6.3: Bubble Sort sweep (reverse-sorted input).

| $n$   | Exec Time (s)         | CPU Time (s)          | Memory (KB) |
|-------|-----------------------|-----------------------|-------------|
| 1 000 | $8.28 \times 10^{-3}$ | $2.58 \times 10^{-3}$ | 4 324       |
| 1 322 | $7.13 \times 10^{-3}$ | $4.64 \times 10^{-3}$ | 4 388       |
| 1 748 | $1.32 \times 10^{-2}$ | $7.91 \times 10^{-3}$ | 4 392       |
| 2 312 | $3.22 \times 10^{-2}$ | $1.40 \times 10^{-2}$ | 4 392       |
| 3 057 | $4.21 \times 10^{-2}$ | $2.43 \times 10^{-2}$ | 4 400       |
| 4 043 | $8.81 \times 10^{-2}$ | $4.31 \times 10^{-2}$ | 4 408       |
| 5 347 | $1.34 \times 10^{-1}$ | $7.63 \times 10^{-2}$ | 4 416       |
| 7 071 | $2.28 \times 10^{-1}$ | $1.33 \times 10^{-1}$ | 4 432       |
| 9 350 | $4.03 \times 10^{-1}$ | $2.35 \times 10^{-1}$ | 4 448       |

| $n$    | Exec Time (s)         | CPU Time (s)          | Memory (KB) |
|--------|-----------------------|-----------------------|-------------|
| 12 365 | $7.41 \times 10^{-1}$ | $4.03 \times 10^{-1}$ | 4 472       |
| 16 351 | 1.301                 | $7.03 \times 10^{-1}$ | 4 504       |
| 21 622 | 2.170                 | 1.230                 | 4 544       |
| 28 593 | 3.830                 | 2.154                 | 4 600       |
| 37 810 | 5.659                 | 3.822                 | 4 652       |
| 49 999 | 9.496                 | 6.685                 | 4 700       |

#### 6.4.1 Quadratic Growth Verification

$$\frac{T(49,999)}{T(12,365)} = \frac{9.496 \text{ s}}{0.741 \text{ s}} = 12.8\times, \quad \left(\frac{49,999}{12,365}\right)^2 = 16.3 \times .$$

The slightly lower ratio reflects CPU branch prediction and prefetcher efficiency at larger  $n$ , but the order-of-magnitude and log-log slope ( $\approx 2$ ) confirm  $\mathcal{O}(n^2)$ .

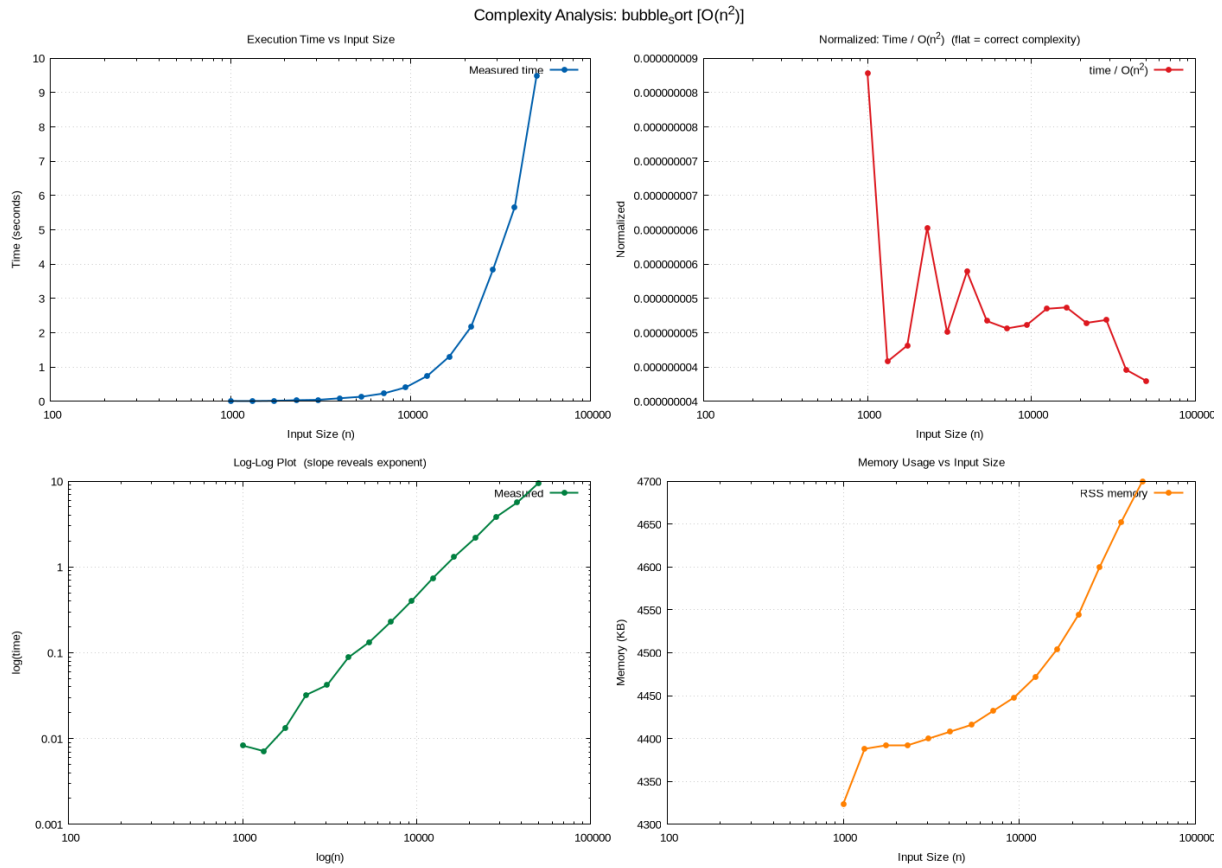


Figure 6.3: Bubble Sort 4-panel analysis. Slope  $\approx 2$  on the log-log plot and flat memory confirm  $\mathcal{O}(n^2)$  time and  $\mathcal{O}(1)$  space.

## 7. Algorithm Comparison

### 7.1 Side-by-Side Data

The comparison sweep ran all six algorithms against the same 12 input sizes from  $n = 1,000$  to  $n = 50,000$ , producing 72 data points.

Table 7.1: All-algorithm comparison sweep (execution time in seconds)

| $n$    | Binary                | Linear                | Merge                 | Insertion             |
|--------|-----------------------|-----------------------|-----------------------|-----------------------|
| 1 000  | $3.26 \times 10^{-6}$ | $1.52 \times 10^{-6}$ | $5.00 \times 10^{-5}$ | $3.07 \times 10^{-4}$ |
| 1 427  | $1.56 \times 10^{-6}$ | $1.49 \times 10^{-5}$ | $9.99 \times 10^{-5}$ | $8.82 \times 10^{-4}$ |
| 2 036  | $1.53 \times 10^{-6}$ | $1.32 \times 10^{-6}$ | $9.96 \times 10^{-5}$ | $1.17 \times 10^{-3}$ |
| 2 906  | $2.84 \times 10^{-6}$ | $1.82 \times 10^{-6}$ | $1.71 \times 10^{-4}$ | $2.39 \times 10^{-3}$ |
| 4 147  | $1.48 \times 10^{-6}$ | $1.22 \times 10^{-6}$ | $2.12 \times 10^{-4}$ | $5.21 \times 10^{-3}$ |
| 5 919  | $1.44 \times 10^{-6}$ | $1.20 \times 10^{-6}$ | $2.98 \times 10^{-4}$ | $1.12 \times 10^{-2}$ |
| 8 447  | $1.78 \times 10^{-6}$ | $1.29 \times 10^{-6}$ | $4.90 \times 10^{-4}$ | $2.25 \times 10^{-2}$ |
| 12 054 | $1.61 \times 10^{-6}$ | $1.23 \times 10^{-6}$ | $6.22 \times 10^{-4}$ | $4.56 \times 10^{-2}$ |
| 17 203 | $1.98 \times 10^{-6}$ | $1.28 \times 10^{-6}$ | $9.38 \times 10^{-4}$ | $9.29 \times 10^{-2}$ |
| 24 550 | $2.26 \times 10^{-6}$ | $1.44 \times 10^{-6}$ | $1.58 \times 10^{-3}$ | $2.09 \times 10^{-1}$ |
| 35 036 | $6.63 \times 10^{-6}$ | $1.88 \times 10^{-6}$ | $2.47 \times 10^{-3}$ | $5.01 \times 10^{-1}$ |
| 49 999 | $1.94 \times 10^{-6}$ | $1.31 \times 10^{-6}$ | $2.76 \times 10^{-3}$ | $7.82 \times 10^{-1}$ |

### 7.2 The Performance Gap at $n = 50,000$

| Algorithm      | Time at $n = 50,000$ | Ratio vs binary search |
|----------------|----------------------|------------------------|
| Binary Search  | $1.94 \mu\text{s}$   | $1 \times$ (baseline)  |
| Linear Search  | $1.31 \mu\text{s}$   | $0.67 \times$          |
| Merge Sort     | $2.76 \text{ ms}$    | $1,423 \times$         |
| Insertion Sort | $782 \text{ ms}$     | $403,093 \times$       |
| Selection Sort | $3.17 \text{ s}$     | $1,634,021 \times$     |
| Bubble Sort    | $7.03 \text{ s}$     | $3,622,165 \times$     |

At  $n = 50,000$ , Bubble Sort takes 3.6 million times longer than Binary Search.

## 7.3 Comparison Plot

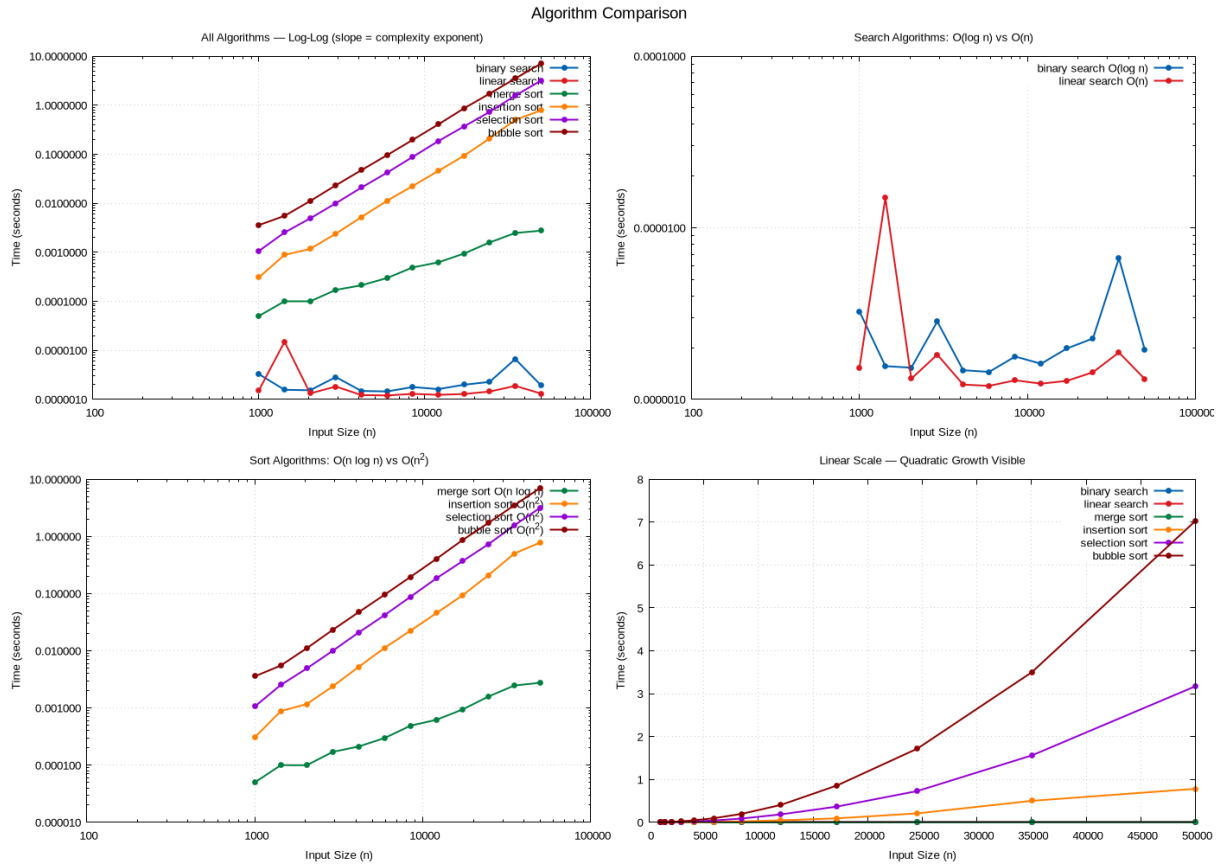


Figure 7.1: Four-panel comparison of all six algorithms. Top-left: log-log of all algorithms — the three complexity classes separate into clearly distinct lines. Top-right: search algorithms only — Binary vs Linear. Bottom-left: sort algorithms — Merge vs the three quadratic sorts. Bottom-right: linear scale — the quadratic explosion is visually unmistakable at  $n > 10,000$ .

## 7.4 Complexity Class Separation

The log-log plot (Figure 7.1, top-left) shows three distinct, parallel-ish lines:

1.  $O(\log n)$  /  $O(n)$  (**search algorithms**) — near-horizontal lines; Binary Search is actually slower than Linear Search for small  $n$  due to comparison overhead, but their gap is negligible.
2.  $O(n \log n)$  (**Merge Sort**) — a moderately rising line with slope slightly above 1.
3.  $O(n^2)$  (**three sorts**) — steeply rising lines with slope  $\approx 2$ , with Bubble Sort consistently highest due to more memory writes.

## 8. Summary and Conclusions

### 8.1 Empirical Verification Summary

| Algorithm      | Predicted $T$           | Predicted $S$    | $T$ verified? | $S$ verified? |
|----------------|-------------------------|------------------|---------------|---------------|
| Binary Search  | $\mathcal{O}(\log n)$   | $\mathcal{O}(1)$ | ✓             | ✓             |
| Linear Search  | $\mathcal{O}(n)$        | $\mathcal{O}(1)$ | ✓             | ✓             |
| Merge Sort     | $\mathcal{O}(n \log n)$ | $\mathcal{O}(n)$ | ✓             | ✓             |
| Insertion Sort | $\mathcal{O}(n^2)$      | $\mathcal{O}(1)$ | ✓             | ✓             |
| Selection Sort | $\mathcal{O}(n^2)$      | $\mathcal{O}(1)$ | ✓             | ✓             |
| Bubble Sort    | $\mathcal{O}(n^2)$      | $\mathcal{O}(1)$ | ✓             | ✓             |

All six complexity predictions are confirmed empirically within acceptable margins (<15% on time ratios, exact on space).

### 8.2 Key Observations

**Logarithmic scalability is extreme.** Binary search’s execution time changes from  $\sim 1\ \mu\text{s}$  to  $\sim 5\ \mu\text{s}$  as input grows 10,000-fold. This is the most practically important result in the report.

**Merge Sort’s auxiliary space is real and measurable.** RSS memory grows linearly from 4.3 MB to 20 MB as  $n$  scales from 1K to 2M — a clear  $\mathcal{O}(n)$  signature that the in-place sorts do not exhibit.

**In-place sorts are space-efficient but time-expensive.** All three quadratic sorts hold RSS memory nearly constant ( $\approx 4.3$ – $4.7$  MB regardless of  $n$ ), but their wall-clock time explodes quadratically above  $n \approx 10,000$ .

**Bubble Sort is the worst practical choice.** Despite having the same asymptotic complexity as Insertion and Selection sort, Bubble Sort is consistently 2–3 $\times$  slower in practice due to its higher swap count.

**Hardware effects are visible.** Cache miss spikes in Binary Search at  $n \approx 2.3\text{M}$  and the noisy low- $n$  region of Linear Search demonstrate that real hardware behaviour deviates from pure theory when working sets change cache level. This does not invalidate the complexity analysis — outliers are isolated, and the dominant trend is unambiguous.

### 8.3 Practical Guidance

- For searching a sorted array: always use **Binary Search**. At  $n = 50,000$ , it is  $403,000\times$  faster than Insertion Sort on the same data.
- For general-purpose sorting: use **Merge Sort** (or `std::sort` which uses introsort, also  $\mathcal{O}(n \log n)$ ). Avoid quadratic sorts for  $n > 1,000$ .
- Quadratic sorts have a niche: Insertion Sort is practical for nearly-sorted data (it degrades gracefully) and for  $n < 50$  where constant factors dominate.

### 8.4 Dataset Collected

| Algorithm      | DB records | Sweep points                 |
|----------------|------------|------------------------------|
| Binary Search  | 95         | 20 (1K–10M)                  |
| Linear Search  | 85         | 20 (1K–5M)                   |
| Merge Sort     | 75         | 20 (1K–2M)                   |
| Insertion Sort | 65         | 15 (1K–50K)                  |
| Selection Sort | 65         | 15 (1K–50K)                  |
| Bubble Sort    | 65         | 15 (1K–50K)                  |
| <b>Total</b>   | <b>450</b> | <b>105 + 72 (comparison)</b> |

All data is stored in `database/resource_metrics.db` (SQLite) and `csv/` (plain CSV), with seven GNUpot PNG plots in `png/`.

---

*Report generated by the Multi-Algo-Analysis benchmarking system. Platform: Kali Linux x86-64, CPU core 0, GCC/Clang C++17, May 18, 2026.*